

Ontology-Oriented Software Development

Why a forty-year-old idea failed, what changed, and how a conventional e-commerce platform is translated to it — worked end to end.

Case study: Order-Up — cold-chain grocery delivery

Stack: Flutter • Node/Express • MySQL • Firebase • S3 • Razorpay

Personas: consumer • store manager • delivery agent • platform manager

ABSTRACT

Conventional service architectures store data and behaviour, but not *meaning*. Business definitions — what counts as an active store, a breached delivery, an underperforming seller — exist only as incidental properties of whichever query happens to encode them. Each consuming surface re-derives them independently, and nothing detects when those derivations disagree.

This has always been a real cost, but never one large enough to justify the remedy. Formal ontology was proposed for exactly this problem in the 1990s and 2000s and was correctly rejected by industry: the tooling was academic, the formalism was expensive, and critically, nothing existed that could consume an ontology which could not more cheaply consume hand-written SQL.

Language models change that calculation. A model generating SQL against forty tables is unverifiable; a model selecting a named concept with typed parameters is testable, loggable and gateable. The ontology stops being documentation and becomes the tool schema. This paper argues the shift is economic rather than theoretical, and demonstrates it by translating a real production codebase — including seven defects found during the analysis that are themselves symptoms of the missing layer.

Contents

Right-click and choose “Update Field” to populate page numbers.

1. Introduction.....	3
2. The problem: meaning has no home.....	3
2.1 A worked instance	3
2.2 Why this is structural	4
2.3 The N×M tax.....	4
3. What an ontology is, and is not.....	5
4. History	5
4.1 Knowledge representation, 1966–1993	5
4.2 The Semantic Web bet, 1999–2008	6
4.3 Where it actually worked.....	6
4.4 Why industry passed	6
5. What changed.....	7
6. Case study: Order-Up.....	8
6.1 The system as it stands	8
6.2 Findings from the codebase.....	8
6.3 Target architecture	10
6.4 Entities and source mapping.....	10
6.5 Derived concepts	11
6.6 The semantic layer.....	12
6.7 Worked example: ColdChainBreach end to end.....	12
6.8 Worked example: negotiated pricing	13
6.9 Multilingual as an ontology concern	13
6.10 What moves, what doesn’t.....	14
6.11 Migration in phases	14
7. The determinism boundary	15
8. Risks, costs, and when not to adopt.....	16
9. Conclusion.....	17
Appendix A. Concept catalogue	17

1. Introduction

Most systems know what they store and what they do. Very few know what they mean.

That sounds like a philosophical complaint. It is an operational one. In any system of moderate age, the answer to a question like “is this store live?” or “did this delivery fail?” is not written down anywhere. It is implied — separately and independently — by every query that happens to need it. The definitions are real, load-bearing, and invisible.

For thirty years this was a tolerable inefficiency. Engineers absorbed the definitions into their heads, wrote the query, shipped the feature. The remedy on offer — formal ontology, with its description logics and triple stores and W3C recommendation stack — cost more than the disease. Industry looked at it, and passed. That was the right call.

What has changed is not the theory. It is that a new class of consumer has arrived which cannot work around the absence of an explicit model. A language model handed a schema of forty tables and asked to answer a business question will produce SQL that is plausible, occasionally correct, and fundamentally unverifiable. The same model handed a catalogue of named, typed concepts will produce a call you can validate, log, replay, and refuse.

This paper makes that argument in three parts: the problem as it appears in real code (§2), the history of why the obvious fix was rejected (§3–§4), and the economic shift that reopens it (§5). It then spends the majority of its length on a single worked case study (§6): translating Order-Up, a running cold-chain grocery delivery platform, into an ontology-oriented architecture — with the migration sequenced so that nothing is rewritten and nothing is taken offline.

2. The problem: meaning has no home

2.1 A worked instance

Order-Up is a delivery platform where customers order perishable goods from nearby stores, and refrigerated trucks carry them. It has an Express API over MySQL, a Flutter client serving three personas, and a web portal for the fourth. It is a perfectly ordinary system, competently built.

Consider one question: **is a given store visible to a given customer?** The customer-facing browse endpoint answers it like this:

```
// app/models/producer.model.js - Producer.list
SELECT ... (ST_Distance_Sphere(point(cust_long, cust_lat),
                                point(longitude, latitude))/1000) as dist
FROM users_tb ...
  INNER JOIN producer_list pl ON x.producerid = pl.producer_id
WHERE x.dist <= distance AND x.user_type = 1
```

Three separate business rules are encoded in that fragment, none of them named:

- A store is only visible if it has a *manager account* — the `INNER JOIN` to `users_tb` silently drops any store without one.
- Each store carries its *own* delivery radius, in a column called `distance` on the user row.
- Store identity lives in `producer_list`, but store *availability* lives on the manager’s user record.

Meanwhile, the platform dashboard answers a closely related question — how many participants are active — with its own, unrelated expression:

```
// app/models/admin.model.js — Admin.dashboard_stats
(SELECT COUNT(*) FROM users_tb WHERE user_type = 2 AND status = 0) as total_delivery_agents,
(SELECT COUNT(*) FROM users_tb WHERE user_type = 0 AND status = 0) as total_customers
```

Note that the browse query never checks `status` at all. So a store whose manager account has been deactivated still appears to customers, while the dashboard correctly reports that manager as inactive. Two views of the platform, disagreeing, with no test that could catch it — because there is no shared definition for a test to check against.

This is not a code quality problem. Both queries are competent. It is a meaning problem, and refactoring will not fix it.

2.2 Why this is structural

It is tempting to read the above as carelessness. It is not. The architecture offers nowhere to put the definition. A controller can hold logic; a model can hold a query; neither can hold the *statement* “a store is visible when its manager is active and the customer is inside its radius” in a form that other code can reference.

So the definition goes into the only durable medium available: the heads of the engineers who wrote it, plus a comment if you are fortunate. In Order-Up’s case there is in fact a comment, in `schema.sql`, and it is instructive:

```
-- (producer_list.producer_id is also treated as == the producer's users_tb.user_id
-- in a couple of distance queries in cart.model.js / order.model.js)
```

Someone knew that two different identifiers are conflated in some queries and not others, understood it was dangerous, and had nowhere to record it except a comment in a file that no running code reads. That comment is a fossil of the missing layer.

2.3 The N×M tax

Count the consumers of a single concept. For “underperforming store” in a platform like this, there are at least five: the platform dashboard, the store manager’s own view, a nightly notification job, whatever spreadsheet finance builds quarterly, and — imminently — an agent. Five consumers, five hand-written derivations, five independent opportunities to drift.

The cost is not the initial writing. It is that adding the sixth surface costs $O(\text{number of concepts})$ rather than $O(1)$, and that changing the definition costs $O(\text{number of surfaces})$ with no compiler to

tell you when you missed one. This is the tax that never appears on a roadmap, because it is charged in small increments to every feature forever.

3. What an ontology is, and is not

The canonical definition is Tom Gruber's, from 1993: *an ontology is an explicit specification of a conceptualization*. Unpacked for practitioners, an ontology in this sense has four parts:

- **Entities** — the kinds of thing the domain contains, each mapped to where its data actually lives.
- **Relationships** — how entities connect, including the joins that are currently implicit.
- **Derived concepts** — the computed predicates and measures the business argues about, each defined exactly once.
- **Policy** — who may see what, expressed over entities rather than over endpoints.

Critically, all four are *code*: in the repository, version-controlled, code-reviewed, unit-tested, and imported by the things that use them. An ontology that lives in a modelling tool is a diagram, and diagrams drift.

WHAT THIS IS NOT

Not a triple store — the relational database stays exactly where it is. Not RDF, OWL or SPARQL. Not a reasoner; there is no inference engine at runtime. Not an upper ontology — no BFO, DOLCE or SUMO, and no attempt to model the universe. Not a deliverable produced by an architecture function separate from the teams that ship.

Every item on that list contributed to the failure of the previous attempt. The proposal here is to keep the idea and discard the apparatus.

4. History

The case for an explicit model of meaning is not new, and anyone proposing it should be able to explain why the previous forty years of attempts did not stick.

4.1 Knowledge representation, 1966–1993

Ross Quillian's semantic networks (1966) represented meaning as a labelled graph of associations. Marvin Minsky's frames (1974) added structured stereotypes of situations — the direct ancestor of every class hierarchy since. The expert systems of the following decade turned this into product: MYCIN at Stanford diagnosed bloodstream infections at the level of specialists and was never

clinically deployed; XCON at Digital Equipment Corporation configured VAX orders from 1980 and is widely credited with saving the company tens of millions of dollars annually.

Two projects defined the era’s ambition and its limits. Doug Lenat’s Cyc, begun in 1984, set out to hand-encode common-sense knowledge and is still going four decades later. KL-ONE (1985) and the description logics that followed gave frames a rigorous formal semantics — and in doing so made explicit the trade-off that would dog the field: expressivity and tractable reasoning cannot both be maximised. Gruber’s 1993 definition arrived as the field was consolidating.

4.2 The Semantic Web bet, 1999–2008

YEAR	MILESTONE	INTENT
1999	RDF becomes a W3C Recommendation	Universal subject-predicate-object data model
2001	Berners-Lee, Hendler & Lassila, Scientific American	Agents traversing machine-readable meaning across the web
2004	OWL standardised	Formal ontology language; three dialects trading expressivity against decidability
2008	SPARQL standardised	Query language completing the stack

The stack was coherent and largely worked. Its failure was economic, not technical: the cost of annotation fell on content publishers, while the benefit accrued to whoever consumed the annotations. Outside academia and a few well-funded verticals, that trade never cleared.

4.3 Where it actually worked

Ontologies succeeded in two circumstances, and it is worth being precise about which.

Where curation was funded. The Gene Ontology (1998) remains core biological infrastructure. SNOMED CT and UMLS underpin clinical terminology at national scale. These domains are shared, slow-moving, and had grant funding for full-time curators — an arrangement no product team has.

Where payoff was immediate. schema.org (2011) is almost certainly the most widely deployed ontology ever built, because marking up a page produced a richer search result the same week. Google’s Knowledge Graph (2012) kept the graph and quietly dropped the formalism. Palantir ships an *Ontology* as the central abstraction of its platform. And in analytics, the semantic layer was independently reinvented without any of the philosophy — LookML (2012), then dbt’s metrics layer, Cube, and their peers — solving precisely the “one definition of revenue” problem this paper describes, for the narrow case of dashboards.

Adoption tracked immediate payoff in every case. Never elegance, never completeness.

4.4 Why industry passed

1. **Nothing consumed it.** You modelled the domain, and then wrote the SQL by hand anyway. The return never justified the investment — this is the decisive reason, and the others are secondary.
2. **The formalism trap.** OWL Full is undecidable; OWL DL reasoning is worst-case exponential. Teams spent months on modelling debates — is *Manager* a subclass of *Employee* or a role played by one? — that produced no user-visible change.
3. **Upper-ontology wars.** Cyc versus SUMO versus BFO versus DOLCE consumed enormous effort at maximum abstraction and minimum consequence.
4. **Academic tooling.** Protégé, Jena and Virtuoso competed against Rails and Django scaffolding that produced a working CRUD application in an afternoon.
5. **Triple stores lost on operations.** Slower, stranger to operate, and unfamiliar to every DBA. Nobody was fired for choosing Postgres.
6. **Domain-Driven Design won the argument.** Eric Evans (2003) advised building many small bounded contexts with explicit translation at the seams, rather than one global model. That advice was correct, and it is the strongest intellectual objection to this paper’s proposal. §8 addresses it directly.
7. **Curation never ends.** Ontologies rot. In science, grants paid for curators; in a product organisation, ownership was never assigned.
8. **Written by architects, ignored by engineers.** When the model is produced by a group that does not ship the code, the artifact and the system begin diverging on day one.

5. What changed

Reason 1 above is load-bearing: for three decades the only things capable of consuming an ontology were humans and rule engines, and both had cheaper alternatives. That is no longer true.

	MODEL WRITES SQL	MODEL SELECTS A CONCEPT
Surface area	~23 tables, implicit joins, undocumented conventions	A dozen named concepts with typed parameters
Verifiability	Cannot diff generated SQL against intent	Wrong concept selection is a testable failure
Auditability	Query text in a log, semantics unknown	Concept plus arguments, replayable
Safety near money	None — the model is the last checkpoint	Policy the model cannot see or edit

The consequence is that **the ontology becomes the tool schema**. The same catalogue that gives humans one definition of a breach gives the model its available actions. Two problems, one artifact — which is what finally clears the investment bar.

The economics have inverted in the other direction too. Previously, formalising cost a lot and saved little. Now, *not* formalising is what costs: every agent re-derives the business rules differently, there is no shared definition to evaluate model output against, and there is no path from an answer back to the rule that produced it — which matters increasingly for regulatory reasons as well as engineering ones.

The argument is not that the idea improved. It is that the alternative got expensive.

Finally, the heavyweight apparatus turns out to be unnecessary for this use. A typed concept catalogue with hand-written resolvers over the existing database delivers the benefit. No reasoner, no triple store, no upper ontology. The 2005 answer to “how do I build this” was the thing that sank it; the 2026 answer is ordinary code.

6. Case study: Order-Up

6.1 The system as it stands

Order-Up is a cold-chain grocery delivery platform. Customers browse nearby stores, order perishable goods, and receive them via refrigerated trucks whose temperature and position are telemetered during transit. It is a genuinely interesting domain, because correctness has a physical dimension: a delivery can complete successfully and still have failed.

COMPONENT	DETAIL
API	Node/Express, ~73 endpoints across 11 route modules; controller + model per domain area
Database	MySQL, 23 tables, accessed through the raw <code>mysql</code> driver
Clients	Flutter app (consumer, store manager, delivery agent); separate web portal (platform manager)
Identity	Firebase for authentication; JWT access and refresh tokens issued by the API
Personas	<code>users_tb.user_type</code> — 0 consumer, 1 store manager, 2 delivery agent, 3 platform manager
External	AWS S3 (images, via <code>multiparty</code>), Firebase Cloud Messaging (push), Razorpay (payments)

6.2 Findings from the codebase

The following were identified while reading the source in preparation for this paper. They are included not as criticism — every one is the kind of thing that accumulates in a working system — but

because each is a *symptom of the missing layer*, and together they make the abstract argument concrete.

FINDING 1 • VOCABULARY

The order status vocabulary has holes

`constants.js` defines `ORDER_STATUS` with keys 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14. Four and five are absent. Either two states were retired without migration, or two states exist in data with no name. Nothing in the system can tell you which, because the status vocabulary is a bare object literal rather than a modelled concept with a lifecycle.

FINDING 2 • POLARITY

Active means zero

`auth.js` validates sessions with `where status = 0`. Zero is active; one is deactivated. This is defensible but inverted from most engineers' intuition, is written down nowhere, and is re-typed as a literal in every query that needs it. A single mistyped polarity silently grants or denies access.

FINDING 3 • AUTHORIZATION

Identity is taken from the request body, not the token

The `/uploadImage` handler in `index.js` updates `users_tb.profile_pic` keyed on an `fb_id` read from `req.body` — despite `req.user_id` being available from the verified token. Any authenticated user can therefore overwrite any other user's profile image. The same statement is built by string concatenation, so the parameter is also an injection vector.

FINDING 4 • AUTHORIZATION

Unauthenticated telemetry ingestion

Routes are mounted in two groups either side of `app.use(auth)`. `temperature.routes.js` is mounted *before* it, making `/add_temperature` public. Anyone who can reach the host can submit temperature and position readings for any device id. Cold-chain evidence — the basis for refunds and claims — is therefore unauthenticated.

FINDING 5 • DEFAULTS

Missing claims default to consumer

`auth.js` sets `req.user_type = 0` when a token carries no type. Defaulting to the least-privileged persona is the safe direction, but it is an implicit rule discoverable only by reading the middleware, and it means a malformed token produces a working consumer session rather than an error.

FINDING 6 • IDENTITY

Two identifiers used interchangeably

Per the comment in `schema.sql`, `producer_list.producer_id` is treated as equal to the producer's `users_tb.user_id` in some distance queries but not others. Where the two ever diverge, results are silently wrong rather than erroneous.

FINDING 7 • CONSISTENCY

Two payment providers

The API depends on `razorpay`; the Flutter client depends on `flutter_stripe`. Whichever is vestigial, the system currently carries two payment vocabularies, and no single place defines what a *Payment* is independently of provider.

Findings 3 and 4 are ordinary security bugs and should be fixed on their own merits, regardless of anything in this paper. But note the pattern across all seven: each is a case where a rule that ought to be stated once — *what active means, who may modify a profile, what a valid status is, what identifies a store* — is instead re-expressed ad hoc at each site, with no mechanism to detect disagreement. That is precisely the failure mode an ontology layer exists to remove.

6.3 Target architecture

LAYER	CONTAINS
Experience	Apps, portal, agents — models live here
Semantic	Governed queries, policy — no model, ever
Ontology	Entities, relationships, derived concepts
Data sources	MySQL, Firebase, S3, Razorpay — unchanged
Infrastructure	Runtime + model gateway: hosted or local

Figure 1 – Five layers. The two in the middle are the entire change; the layers above keep their current contracts and the layer below keeps its current schema.

The existing Express routes remain as a thin HTTP shell in the experience layer. The logic currently inside controllers and models migrates downward into ontology definitions and semantic resolvers. Nothing about the database changes.

6.4 Entities and source mapping

The first act of the ontology is unglamorous and valuable: naming what the schema already holds, and recording the joins that are currently tribal knowledge.

ENTITY	BACKED BY	NOTE
Customer	users_tb (user_type 0)	Addresses in user_address
Store	producer_list + users_tb (user_type 1)	Identity and availability live in different tables
Product	product_list, product_img_tb	Gains one new attribute: safe temperature range
Order	order_tb, order_details_id	Table name reads like a column; concept name fixes this
DeliveryLeg	truck_tb, order_history	One truck run carrying one or more orders
DeliveryAgent	users_tb (user_type 2)	
TempReading	temp_location, temp_location_history	Written by the public /add_temperature — see Finding 4
Claim	product_return, order_return_reason_tb	
Review	product_review	Multi-dimensional: app, vehicle, driving, payment, overall
Offer	new	Introduced for negotiated pricing — see §6.8

An entity definition carries its source mapping explicitly, which is where a fact like Finding 6 stops being a comment and becomes enforced code:

```
defineEntity('Store', {
  source: { table: 'producer_list', key: 'producer_id' },
  attributes: {
    id: 'producer_id',
    name: 'producer_name',
    description: 'prod_desc',
  },
  relationships: {
    // The operator lives in a different table, joined on a column whose
    // name gives no hint of the relationship. Stated once, here.
    manager: {
      entity: 'StoreManager',
      join: { table: 'users_tb', from: 'producer_id', to: 'producerid' },
      where: { user_type: USER_TYPE.STORE_MANAGER },
    },
  },
});
```

6.5 Derived concepts

Derived concepts are where the value concentrates. Each is a business predicate or measure, defined once, consumed everywhere.

```
// Replaces three implicit rules from Producer.list, and closes the
// browse/dashboard disagreement described in §2.1.
defineConcept('StoreIsVisibleTo', {
  params: { store: 'Store', customer: 'Customer' },
  rule: (store, customer) =>
    store.manager.isActive // was: never checked
    && distanceKm(customer.location, store.manager.location)
    <= store.manager.deliveryRadiusKm, // was: users_tb.distance
});

// The concept the platform is named for, and which nothing currently computes.
defineConcept('ColdChainBreach', {
  params: { leg: 'DeliveryLeg' },
  requires: ['Product.safeTempRange'], // the one genuinely new attribute
  rule: (leg) => leg.readings.some(r =>
    leg.products.some(p => r.celsius > p.safeTempRange.max
      || r.celsius < p.safeTempRange.min)),
});
```

Note what the second definition reveals. Order-Up records temperature (`/add_temperature`) and reads it back (`/fetch_temprature`), but *no table records what temperature any product should be kept at*. The system has always had the telemetry and never had the standard to judge it against. Writing the concept down is what surfaces the gap — which is a recurring effect of this exercise and, in our view, its most underrated benefit.

6.6 The semantic layer

The semantic layer resolves concepts to queries and enforces policy. It contains no model and no probabilistic behaviour of any kind. Policy expressed over entities replaces per-route guards, which is the structural fix for Findings 3 and 5:

```
definePolicy('Order', {
  [USER_TYPE.CONSUMER]: (u, o) => o.customer.id === u.id,
  [USER_TYPE.STORE_MANAGER]: (u, o) => o.store.id === u.storeId,
  [USER_TYPE.DELIVERY_AGENT]: (u, o) => o.leg.agent.id === u.id,
  [USER_TYPE.PLATFORM_MANAGER]: () => true,
});
```

The difference from `requireAdmin.js` is not cosmetic. A route guard answers “may this caller reach this URL”; an entity policy answers “may this caller see this object”. Only the latter composes — and only the latter can constrain an agent that assembles its own access paths at runtime.

6.7 Worked example: ColdChainBreach end to end

One concept, followed through all five layers.

Data. `temp_location` already receives device id, latitude, longitude and temperature. No schema change.

Ontology. Add `Product.safeTempRange` — the missing standard — and define `ColdChainBreach` as in §6.5. One new column, one new definition.

Semantic. A hand-written resolver turns the concept into a query, and a policy states that a store manager sees breaches on their own legs, an agent sees their own, and the platform manager sees all.

Experience. The concept surfaces four ways, all reading the identical definition:

- A panel on the existing admin dashboard
- A push notification to the customer while the truck is still moving
- Automatic corroboration when a claim is filed — the evidence is already there
- An agent tool, generated directly from the concept definition

```
{
  "name": "cold_chain_breaches",
  "description": "Delivery legs where a temperature reading fell outside the
                 safe range of any product on board.",
  "input_schema": {
    "type": "object",
    "properties": {
      "window_days": { "type": "integer" },
      "store_id":    { "type": "integer" }
    }
  }
}
```

That schema is *derived*, not authored. This is the claim of §5 made concrete: one artifact serves the dashboard, the notification, the claims process and the agent, and none of them can hold a different opinion about what a breach is.

6.8 Worked example: negotiated pricing

Haggling is worth examining because it is the case where letting a model act freely would be most obviously unsafe, and where the layering earns its keep.

The ontology gains an Offer entity (bid, counter, accept, reject) and a derived NegotiableBand. The band is computed deterministically from the store's floor price, current margin, stock age, and the existing urgent_charges and conveyance_tb inputs. The model's role is confined to two genuinely non-deterministic judgements: *where inside the band to land*, and *how to phrase the counter*.

THE GOVERNING RULE

The model proposes. Deterministic code commits. The haggle agent may offer any price it likes; the floor decides whether that offer can be accepted. If the model hallucinates a figure below floor, the semantic layer rejects it before it reaches /place_order — which already validates a discount parameter today, so the commit path exists.

This bounds the model's authority by a rule it can neither see nor edit. It also turns a discount mechanism into an inventory instrument: stock approaching expiry can widen its own band automatically, making negotiation a margin lever rather than pure leakage.

6.9 Multilingual as an ontology concern

Language is contained entirely within the experience layer. The consumer agent detects the incoming language and replies in it; the semantic layer never receives a natural-language string, only typed

concept calls. The single ontology change is language-tagged labels on entities and products, so that spinach, पालक and keera resolve to one Product rather than three catalogue entries.

The practical consequence: adding a language means adding labels and a prompt, not touching a single query. Compare the current state, where /sku_gallery performs a literal string match and a query in any other language simply returns nothing.

6.10 What moves, what doesn't

TODAY	LANDS IN	CHANGE
*.routes.js, express-validator checks	Experience	Stays — thin HTTP shell
Controller and model business logic	Ontology + Semantic	This is the migration
auth.js, requireAdmin.js	Semantic	Policy over entities, not per route
constants.js status maps	Ontology	Becomes a modelled vocabulary — Findings 1, 2
MySQL schema	Data source	Unchanged
Firebase, S3, Razorpay	Data source	Unchanged, wrapped
Flutter app, admin portal	Experience	Keep working throughout

6.11 Migration in phases

The migration is deliberately incremental. No phase requires the previous one to be complete across the whole domain, and every phase is independently abandonable.

PHASE 0 — ONE WEEK

Name what already exists

Replace magic literals with a modelled vocabulary: user types, the status polarity from Finding 2, and the gapped order-status set from Finding 1. Zero behavioural change; the entire value is that the definitions acquire one home and one spelling.

Exit: no bare user_type or status literals remain in models or middleware.

PHASE 1 — TWO TO THREE WEEKS

One concept, end to end

Implement ColdChainBreach per §6.7: the new product attribute, the concept, one resolver, and exactly one new read path. Touch no existing route.

Exit: the dashboard panel is live and its number is reproducible from the definition alone.

PHASE 2

Move a contested definition

Implement `StoreIsVisibleTo` and route *both* the customer browse path and the platform dashboard through it. This is the first phase that changes existing behaviour — deliberately, because it resolves the §2.1 disagreement.

Exit: no query outside the semantic layer decides store visibility.

PHASE 3

Consolidate policy

Move authorization from route guards to entity policy, closing Findings 3 and 5 structurally rather than individually.

Exit: every read of an `Order` passes through one policy function.

PHASE 4

First agent

Expose the concept catalogue as tool schemas and ship one read-only agent — platform analytics is the lowest-risk choice, since it commits nothing.

Exit: an eval set exists and the agent scores against it before every prompt or model change.

PHASE 5

New capability

Only now attempt things that were previously impractical: negotiated pricing, multilingual consumer conversation, cold-chain-aware stop sequencing.

Exit: each ships with a deterministic gate on every state-changing path.

If Phase 1 does not pay for itself, the total loss is one concept — not a quarter, and not a rewrite.

7. The determinism boundary

The single most important rule in an architecture that mixes deterministic and probabilistic components is where the boundary sits.

DETERMINISTIC – NEVER A MODEL	NON-DETERMINISTIC – THE MODEL’S ROLE
Breach detection: reading against threshold	Conversation in the customer’s language
Price floor and maximum discount	Where inside the band to counter, and its phrasing
Who may see which orders	Reading free text into a claim category
Payment capture, refunds, state transitions	Demand forecasting per delivery slot
Invoice totals, fees, taxes	Stop sequencing under competing constraints
Any number reaching an SLA report	Ranking ambiguous claims for human review

Two consequences follow. First, a deterministic rule must never be replaced by a model for convenience: if a model decides what a breach is, last quarter’s SLA report cannot be reproduced and a refund decision cannot be defended. Second, every model output touching money, safety, access or a reported number must pass a deterministic gate before it becomes real.

This also changes how the work is specified. A deterministic component has acceptance criteria and passes or fails. A non-deterministic one has a rubric, a target success rate, an eval set that is re-run on every prompt or model change, and a defined fallback. “It works” ceases to be a complete specification for half the product.

8. Risks, costs, and when not to adopt

The DDD objection. The strongest argument against this proposal is Evans’: do not build one global model; build bounded contexts with explicit translation between them. We accept it. The distinction is that an ontology in this sense is shared *vocabulary*, not shared *implementation*. Contexts continue to own their resolvers and their storage; what they share is agreement on what a word denotes. A context may legitimately decline to implement a concept — what it may not do is implement it differently under the same name.

Ownership. Every historical failure in §4.4 traces back to unowned artifacts. If the concept catalogue is not owned jointly by engineering and product, held in the repository, versioned and tested like any other code, it will rot exactly as its predecessors did. An ontology maintained by a separate architecture function is a diagram with extra steps.

Query performance. An abstraction over SQL invites generated SQL, and generated SQL is where semantic layers historically became operationally unacceptable. Resolvers should be hand-written and individually optimised. The ontology constrains *meaning*, not execution plans.

Over-modelling. The failure mode is modelling the universe. Model only concepts with more than one consumer and an actual disagreement to settle. A concept used in one place should stay as a function.

WHEN NOT TO DO THIS

One surface, one team, no agents — do not. A CRUD application with no shared derived concepts — do not. Fewer than roughly three consumers of the same definition — the abstraction will not repay its cost. Nobody willing to own the catalogue — it will rot.

This is a real and ongoing expense. It is justified by multiple consumers, or by agents. It is not justified by taste.

9. Conclusion

The idea that systems should carry an explicit model of what they mean is forty years old and has failed repeatedly. Those failures had valid causes, and the strongest — that nothing consumed the model which could not more cheaply consume hand-written SQL — held right up until models arrived that cannot work around its absence.

The correct response is not to revive the apparatus that sank the earlier attempts. It is to keep the concept catalogue, drop the formalism, and let meaning become an ordinary versioned artifact in the repository: reviewed, tested, and imported by every surface that depends on it.

The Order-Up analysis illustrates both the cost of not doing this and the modesty of the first step. A system that measures temperature on every delivery has never been able to say whether a delivery was too warm, because no definition existed to hold the standard. One concept, one new column, one resolver, and one read path closes that — while the database, the mobile app and every existing endpoint carry on unchanged.

The deterministic core is what allows the probabilistic edge to move quickly. Definitions first, agents second.

Appendix A. Concept catalogue

The initial catalogue proposed for Order-Up. Concepts are listed with the surfaces that would consume them — the count in the final column is the adoption test from §8.

CONCEPT	DEFINITION	CONSUMERS
StoreIsVisibleTo	Manager active, and customer inside the store's own radius	Browse, dashboard, agent — 3
ColdChainBreach	Reading outside the safe range of any product on an active leg	Dashboard, push, claims, agent — 4

CONCEPT	DEFINITION	CONSUMERS
AtRiskDelivery	Thermal headroom insufficient for remaining stops	Agent app, dispatch, alerts — 3
UnderperformingStore	Return and claim rate above threshold on a trailing window	Dashboard, store view, report, agent — 4
IdleAgent	Delivery agent with no active leg in the current slot	Dispatch, reassignment — 2
NegotiableBand	Floor, margin and stock age resolved to an allowed price range	Haggle agent, pricing, commit gate — 3
ClaimIsCorroborated	Claim whose window overlaps a recorded breach on its leg	Claims, fraud triage, agent — 3

Findings in §6.2 were identified by direct source inspection and are reproducible from the repository.
Findings 3 and 4 are security defects and warrant remediation independently of any architectural change.
Historical claims in §4 reflect published dates for the standards, papers and projects cited.